

Guía completa de Git y GitHub desde cero

Edison Achalma

Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Palabras Claves: keyword1, keyword2

Tabla de contenidos

Introduction	3
1 Guía esencial de Git y GitHub	3
1.1 ¿Cómo funciona Git?	3
1.2 Comandos básicos de Git	3
1.3 Instalación de Git en Ubuntu	3
1.3.1 Método 1: Paquetes predeterminados (rápido y estable)	3
1.3.2 Método 2: Desde la fuente (versión más reciente)	4
1.4 Configuración de claves SSH para GitHub	4
1.4.1 Generar una clave SSH	4
1.4.2 Añadir la clave a ssh-agent	5
1.4.3 Vincular la clave a GitHub	5
1.5 Crear un repositorio local	5
1.6 Clonar un repositorio	5
1.6.1 Clonación básica	5
1.6.2 Clonación superficial	6
1.6.3 Clonar una rama específica	6
1.7 Subir un proyecto a GitHub	6
1.8 Observar el repositorio	7
1.9 Trabajar con ramas	7
2 Guardar temporalmente un trabajo no terminado	8

Edison Achalma  <https://orcid.org/0000-0001-6996-3364>

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; <https://credit.niso.org/>) de la siguiente manera: Edison Achalma: conceptualización, redacción

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Email: elmer.achalma.09@unsch.edu.pe

GUÍA COMPLETA DE GIT Y GITHUB DESDE CERO	2
3 Comparar archivos o ramas	8
3.1 Sincronización	8
4 Conclusión	8
5 Publicaciones Similares	9

Guía completa de Git y GitHub desde cero

1 Guía esencial de Git y GitHub

Esta guía te introduce a los fundamentos de Git y GitHub, desde la instalación hasta la gestión avanzada de proyectos. Ideal tanto para principiantes como para quienes buscan perfeccionar sus habilidades en control de versiones.

Git es un sistema de control de versiones (SCV) esencial para rastrear cambios en el código, colaborar en equipo y experimentar con nuevas ideas mediante ramas. Plataformas como GitHub potencian esta colaboración al hospedar repositorios y facilitar el intercambio de código.

1.1 ¿Cómo funciona Git?

Git organiza los proyectos en tres áreas principales: - **Directorio de trabajo**: Donde editas tus archivos. - **Área de preparación (staging)**: Donde preparas los cambios antes de confirmarlos. - **Directorio Git**: Almacena las instantáneas confirmadas de tu proyecto.

Disponible en Linux, Windows y macOS, Git tiene una curva de aprendizaje inicial, pero su dominio abre un mundo de posibilidades para gestionar proyectos eficientemente.

1.2 Comandos básicos de Git

Aquí tienes los comandos fundamentales:

1. `echo "# Léeme" >> README.md`: Crea un archivo README.md con el texto “# Léeme”.
2. `git init`: Inicia un nuevo repositorio en el directorio actual.
3. `git add [file]`: Añade un archivo al área de preparación.
4. `git commit -m "Primer commit"`: Guarda los cambios con un mensaje descriptivo.
5. `git branch -M main`: Muestra las ramas existentes.
6. `git remote add origin git@github.com:achalmed/repositorio.git`: Vincula el repositorio local con uno remoto en GitHub.
7. `git push -u origin main`: Envía los cambios al repositorio remoto.
8. `git status`: Muestra el estado actual del repositorio.
9. `git log`: Lista el historial de commits.
10. `git diff`: Compara cambios no confirmados con el último commit.
11. `git checkout [branch]`: Cambia a una rama específica.
12. `git merge [branch]`: Fusiona una rama con la actual.
13. `git config --global user.name "tu-nombre"`: Configura tu nombre de usuario.
14. `git config --global user.email "tu-email@example.com"`: Configura tu correo.

1.3 Instalación de Git en Ubuntu

1.3.1 Método 1: Paquetes predeterminados (rápido y estable)

1. Verifica si Git está instalado:

```
git --version
```

Ejemplo de salida: `git version 2.34.1`

2. Si no está instalado, actualiza e instala con APT:

```
sudo apt update
sudo apt install git
```

3. Configura tu identidad:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu.correo@example.com"
```

4. Verifica la configuración:

```
git config --list
```

1.3.2 Método 2: Desde la fuente (versión más reciente)

1. Instala las dependencias:

```
sudo apt update
sudo apt install libz-dev libssl-dev libcurl4-gnutls-dev libexpat1-dev gettext c
```

2. Descarga y descomprime la versión deseada (ejemplo: 2.34.1):

```
mkdir tmp && cd tmp
curl -o git.tar.gz https://mirrors.edge.kernel.org/pub/software/scm/git/git-2.34
tar -zxvf git.tar.gz
cd git-*
```

3. Compila e instala:

```
make prefix=/usr/local all
sudo make prefix=/usr/local install
exec bash
```

4. Confirma la instalación:

```
git --version
```

1.4 Configuración de claves SSH para GitHub

1.4.1 Generar una clave SSH

1. Verifica claves existentes:

```
ls -al ~/.ssh
```

Si no hay claves, crea el directorio: `mkdir ~/.ssh`.

2. Genera un par de claves:

```
ssh-keygen -t rsa -b 4096 -C "tu.email@example.com"
```

Acepta el nombre predeterminado y añade una contraseña (opcional).

1.4.2 Añadir la clave a ssh-agent

1. Inicia el agente:

```
eval "$(ssh-agent -s)"
```

2. Añade la clave privada:

```
ssh-add ~/.ssh/id_rsa
```

1.4.3 Vincular la clave a GitHub

1. Copia la clave pública:
 - Linux/Mac: `cat ~/.ssh/id_rsa.pub`
 - Windows: `clip < ~/.ssh/id_rsa.pub`
2. En GitHub, ve a *Settings > SSH and GPG keys > New SSH key*, pega la clave y guárdala.
3. Prueba la conexión:

```
ssh -T git@github.com
```

Resultado esperado: `Hi tu_usuario! You've successfully authenticated...`

1.5 Crear un repositorio local

1. Inicia un repositorio:

```
git init [nombre-del-proyecto]
```

2. Añade archivos y haz un commit:

```
git add .  
git commit -m "Primer commit"
```

1.6 Clonar un repositorio

1.6.1 Clonación básica

Clona un repositorio remoto:

```
git clone https://github.com/usuario/repositorio.git
```

Clonación en carpeta específica

```
git clone https://github.com/usuario/repositorio.git /ruta/deseada
```

1.6.2 Clonación superficial

Solo las últimas n confirmaciones:

```
git clone --depth=1 https://github.com/usuario/repositorio.git
```

1.6.3 Clonar una rama específica

```
git clone --branch=nombre-rama https://github.com/usuario/repositorio.git
```

1.7 Subir un proyecto a GitHub

1. Crea un repositorio en GitHub (público o privado).
2. En tu proyecto local:

Con SSH

```
echo "# examen" >> README.md
git init
git add .
git commit -m "Primer commit"
git branch -M main
git remote add origin git@github.com:usuario/repositorio.git
git push -u origin main
```

Con HTTPS

```
echo "# examen" >> README.md
git init
git add .
git commit -m "Primer commit"
git branch -M main
git remote add origin https://github.com/usuario/repositorio.git
git push -u origin main
```

3. Si aparece el error remote origin already exists:

```
git remote rm origin
```

Luego repite el paso 2.

1.8 Observar el repositorio

- `git status`: Muestra el estado actual.
- `git diff`: Compara cambios no confirmados.
- `git log`
- `git log --graph`
- `git log --graph --pretty=oneline`
- `git log --graph --decorate --all --oneline`
- `git config --global alias.tree "log --graph --decorate --all --oneline"`
- `git tree`
- `git log --oneline`: Historial compacto. Ejemplo:

```
7e320e8 update
```

- `git blame [archivo]`: Autores y fechas de cambios.
- `touch .gitignore`

1.9 Trabajar con ramas

1. Crea y cambia a una rama:

Navegando en ramas - `git checkout holagit.py` - `git log` - `git checkout 707c7f864de5e036c54b43df5a1bfa464fb4d9ba` - `git tree` - `git checkout 380beab`
 Creando ramas

```
git checkout -b nueva-rama
```

```
git branch login git switch login
```

2. Fusiona ramas:

```
git checkout main
git merge nueva-rama
```

```
git switch login git merge main git tree
corrección de conflicto
```

- `git merge main`
- conflicto tal vez por que se modifico la misma linea en distintos branch
- corregir el error
- `git add archivo_conflicto.py`
- `git commit -m "correccion conflicto"`

3. Reset

- `git tree`
- `git reset --hard af18c2a`
- `git log`
- `git reflog log completo`
- `git checkout 380beab`

3. Etiqueta un commit: (Para versiones)

```
git tag v1.0.0
git push origin main --tags
```

- `git tag clase_1`
- `git log o git tree`
- `git tag` Listado de tags
- `git checkout tags/clase_1`
-

2 Guardar temporalmente un trabajo no terminado

```
git status
git switch main
git stash
git stash list
git switch main
Hago los cambios
git switch login
git stash pop
git add .
git commit -m "Login v2"
git stash drop
```

3 Comparar archivos o ramas

```
git diff login
```

3.1 Sincronización

- `git fetch origin`: Descarga cambios remotos sin fusionarlos.
- `git pull origin main`: Descarga y fusiona cambios.
- `git push origin main`: Envía cambios locales al remoto.

4 Conclusión

Dominar Git y GitHub es clave para gestionar proyectos de desarrollo. Practica estos comandos y consulta `git --help` para más detalles.

5 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1.  [Comandos De Informacion Windows](#)
2.  [Adb](#)
3.  [Limpieza Y Optimizacion De Pc](#)
4.  [Usando Apk En Window 11](#)
5.  [Gestionar Versiones De Jdk En Kubuntu](#)
6.  [Instalar Tor Browser](#)
7.  [Crear Enlaces Duros O Hard Link En Linux](#)
8.  [Comandos Vim](#)
9.  [Guia De Git Y Github](#)
10.  [00 Primeros Pasos En Linux](#)
11.  [01 Introduccion Linux](#)
12.  [02 Distribuciones Linux](#)
13.  [03 Instalacion Linux](#)
14.  [04 Administracion Particiones Volúmenes](#)
15.  [Atajos De Teclado Y Comandos Para Usar Vim](#)
16.  [Instalando Specitify](#)
17.  [Gestiona Tus Dotfiles Con Gnu Stow](#)

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!